

# Agent Phantom Recovery

## 13-api-specification.md

### Production API Specification

Version: 1.0

Part 4

---

## 43. Realtime Architecture

### Purpose

Agent Phantom Recovery is fundamentally a realtime execution platform.

Traditional applications operate using:

```
Request
↓
Response
```

Agent Phantom Recovery operates using:

```
Task
↓
Execution
↓
Events
↓
Realtime Updates
↓
Workspace Synchronization
```

The frontend should never depend on polling for critical updates.

Realtime communication is mandatory.

---

## 44. Realtime APIs

### Realtime Connection Metadata

#### Endpoint

```
GET /api/v1/realtime/config
```

#### Response

```
{
  "websocket_url": "",
  "protocol_version": "v1"
}
```

---

### Realtime Channel Discovery

#### Endpoint

```
GET /api/v1/realtime/channels
```

#### Response

```
{
  "channels": [
    "task_updates",
    "terminal_stream",
    "browser_stream",
    "memory_updates",
    "agent_updates"
  ]
}
```

---

## 45. WebSocket Protocol

### Connection

```
wss://api.agent-phantom.com/ws
```

---

## Authentication

Immediately after connection:

```
{
  "type": "AUTH",
  "token": "jwt_token"
}
```

---

## Successful Authentication

```
{
  "type": "AUTH_SUCCESS"
}
```

---

## Failed Authentication

```
{
  "type": "AUTH_FAILED"
}
```

---

# 46. WebSocket Channels

## Task Updates

Channel:

```
task_updates
```

---

## Example Event

```
{
  "type": "TASK_STATUS_CHANGED",
  "task_id": "",
  "status": "ANALYZING"
}
```

---

## Execution Updates

Channel:

execution\_updates

---

### Example Event

```
{
  "type": "EXECUTION_STARTED",
  "execution_id": ""
}
```

---

## Agent Updates

Channel:

agent\_updates

---

### Example Event

```
{
  "type": "REASONER_STARTED",
  "task_id": ""
}
```

---

## Memory Updates

Channel:

memory\_updates

---

## Browser Updates

Channel:

browser\_stream

---

## Terminal Updates

Channel:

terminal\_stream

---

# 47. Terminal Streaming Protocol

## Purpose

Support live command execution.

---

## Example

User executes:

pytest

Terminal should stream:

```
{
  "type": "TERMINAL_OUTPUT",
  "session_id": "",
  "data": "Running tests..."
}
```

---

Additional output:

```
{
  "type": "TERMINAL_OUTPUT",
  "session_id": "",
  "data": "25 tests passed"
}
```

---

Completion:

```
{
  "type": "TERMINAL_COMPLETED",
  "exit_code": 0
}
```

---

## 48. Browser Streaming Protocol

### Purpose

Support Antigravity Browser Workspace.

---

### Navigation Event

```
{
  "type": "BROWSER_NAVIGATION",
  "url": ""
}
```

---

### DOM Observation

```
{
  "type": "DOM_OBSERVATION",
  "summary": ""
}
```

---

### Screenshot Event

```
{
  "type": "SCREENSHOT_READY",
  "artifact_id": ""
}
```

---

## 49. Execution Timeline APIs

### Purpose

Provide complete execution visibility.

---

### Get Timeline

#### Endpoint

```
GET /api/v1/timeline/{taskId}
```

---

#### Response

```
{
  "events": []
}
```

---

### Timeline Event Structure

```
{
  "timestamp": "",
  "source": "REASONER",
  "event": "ROOT_CAUSE_FOUND"
}
```

---

## 50. Event APIs

Events are the backbone of observability.

---

### List Events

#### Endpoint

```
GET /api/v1/events
```

---

## Task Events

### Endpoint

```
GET /api/v1/tasks/{taskId}/events
```

---

## Event Details

### Endpoint

```
GET /api/v1/events/{eventId}
```

---

# 51. Event Types

## Task Events

```
TaskCreated  
TaskStarted  
TaskPaused  
TaskCompleted  
TaskFailed
```

---

## Planning Events

```
PlanGenerated  
PlanUpdated  
PlanRejected
```

---

## Reasoning Events

```
EvidenceCollected  
AnalysisStarted  
RootCauseFound  
PatchGenerated
```

---



## Verification Events

```
VerificationStarted  
VerificationPassed  
VerificationFailed
```

---

## Tool Events

```
ToolStarted  
ToolCompleted  
ToolFailed
```

---

## Browser Events

```
PageLoaded  
NavigationCompleted  
DOMCaptured
```

---

## Terminal Events

```
CommandStarted  
CommandCompleted  
ProcessKilled
```

---

# 52. Monitoring APIs

## Purpose

Operational visibility.

---

## System Health

### Endpoint

```
GET /api/v1/monitoring/health
```

---

## Response

```
{  
  "status": "HEALTHY"  
}
```

---

## Service Health

### Endpoint

```
GET /api/v1/monitoring/services
```

---

### Response

```
{  
  "planner": "UP",  
  "reasoner": "UP",  
  "verifier": "UP"  
}
```

---

## Metrics

### Endpoint

```
GET /api/v1/monitoring/metrics
```

---

## Trace Lookup

### Endpoint

```
GET /api/v1/monitoring/traces/{traceId}
```

---

## 53. Admin APIs

### Purpose

Platform administration.

---

### Active Tasks

#### Endpoint

GET /api/v1/admin/tasks

---

### Running Executions

#### Endpoint

GET /api/v1/admin/executions

---

### Agent Statistics

#### Endpoint

GET /api/v1/admin/agents

---

### Queue Status

#### Endpoint

GET /api/v1/admin/queues

---

### System Statistics

#### Endpoint

GET /api/v1/admin/statistics

---

---

## 54. Error Specification

### Philosophy

Errors must be machine-readable and human-readable.

---

### Standard Error Structure

```
{
  "success": false,
  "error": {
    "code": "",
    "message": "",
    "details": {}
  }
}
```

---

## 55. Error Codes

### Authentication

```
AUTH_INVALID_TOKEN
AUTH_EXPIRED_TOKEN
AUTH_FORBIDDEN
```

---

### Repository

```
REPOSITORY_NOT_FOUND
REPOSITORY_INDEXING_FAILED
```

---

### Task

```
TASK_NOT_FOUND
TASK_FAILED
TASK_BLOCKED
```

---

## Tool

TOOL\_NOT\_AVAILABLE  
TOOL\_TIMEOUT  
TOOL\_EXECUTION\_FAILED

---

## Memory

MEMORY\_NOT\_FOUND  
MEMORY\_RETRIEVAL\_FAILED

---

## Internal

INTERNAL\_ERROR  
SERVICE\_UNAVAILABLE

---

# 56. Rate Limiting

## Purpose

Protect infrastructure.

---

## Auth Endpoints

10 requests/minute

---

## Task APIs

100 requests/minute

---

## Search APIs

60 requests/minute

---

## Tool Execution APIs

30 requests/minute

---

## WebSocket Connections

Maximum 10 active connections per user

---

# 57. Internal Service Communication

## Philosophy

External APIs use REST.

Internal services use event-driven communication.

---

## Service Flow

```
Orchestrator
↓
Planner
↓
Reasoner
↓
Verifier
↓
Execution
```

---

## Internal Message Format

```
{
  "event_id": "",
  "event_type": "",
  "payload": {}
}
```

---

## 58. Queue APIs

### Queue Status

#### Endpoint

```
GET /api/v1/admin/queues
```

---

### Queue Metrics

#### Endpoint

```
GET /api/v1/admin/queues/metrics
```

---

#### Example

```
{
  "planning_queue": 3,
  "analysis_queue": 7,
  "verification_queue": 1
}
```

---

## 59. Security Requirements

### Mandatory Controls

Every endpoint must support:

- JWT validation

- Ownership validation
  - Audit logging
  - Request tracing
- 

## Sensitive Operations

Require additional verification.

Examples:

```
Repository Deletion
Memory Deletion
Task Cancellation
Admin Operations
```

---

## 60. Audit APIs

### Purpose

Provide traceability.

---

### Audit Logs

#### Endpoint

```
GET /api/v1/audit
```

---

### Task Audit Logs

#### Endpoint

```
GET /api/v1/tasks/{taskId}/audit
```

---

### Audit Record

```
{
  "actor": "",
```

---



```
"action": "",  
"timestamp": ""  
}
```

---

## 61. Future API Extensions

The architecture should support future additions without breaking existing contracts.

---

### Multi-Agent Expansion

Future endpoints:

```
/agents/researcher  
/agents/reviewer  
/agents/coordinator
```

---

### Collaboration APIs

Future endpoints:

```
/collaboration  
/workspaces  
/teams
```

---

### CI/CD Integration

Future endpoints:

```
/deployments  
/pipelines  
/releases
```

---

### Knowledge Graph APIs

Future endpoints:

```
/knowledge-graph
/architecture-map
```

---

## 62. Antigravity Optimization Requirements

The API layer must optimize for:

---

### Browser Workspace

Realtime browser state.

---

### Terminal Workspace

Persistent command streaming.

---

### Source Code Workspace

Low-latency file retrieval.

---

### Timeline Workspace

Live event synchronization.

---

### Memory Workspace

Fast semantic retrieval.

---

### Agent Workspace

Continuous execution updates.

---

## 63. Production API Summary

The Agent Phantom Recovery API architecture is built around:

```
Tasks
+
Plans
+
Memory
+
Tools
+
Execution
+
Verification
+
Events
+
Realtime Communication
```

rather than simple chat interactions.

The API layer acts as the execution backbone connecting AntigraVity IDE to the orchestration engine, planner, reasoner, verifier, memory system, repository intelligence layer, execution controller, browser workspace, terminal workspace, and future platform extensions.

---

## End of API Specification

Status: Complete

Version: 1.0

Document Type: Production API Specification